

oop Encapsulation Assignment

```
1
Class Main{
}
```

a . class ---> class is a blueprint or template that used to create objects.

```
Main myObject = new Main();
```

B. object----> Object is a instance of a class that combine data and methods.

```
Public static void printBubbel(){
}
```

C. Methods--->a block of code that created to perform specific task .

```
Mycar.color= "red";
```

D. Attributes ---->A variable declared in side a class but outside of any method ,constructor or block.

```
String ar;
```

E. Reference variable ----> A variable that store memory address of an object.

```
Int marks = 80;
```

F. Primitive variable -----> these are built-in data types that store simple raw values directly in stack memory rather the storing references to object.

```
Public void printX (int x){
}
```

G. Method parameters ---->The variables defined in a method's declaration that act as a placeholder for the data the method needs to perform its task .

```
Publi static void printX(){
Int x = 369;
}
```

H. local variables ---->these variables are created inside of the methods . as above x has created .

```
Int []ar = new int [3];
System.out.print(ar[0]+",");
System.out.print(ar[1]+",");
System.out.print(ar[2]);
```

Output ---->0,0,0

I. Default values -----> Like above every data type has a default value . in above the default value of the int ---> 0. Those default values appear when there is no value assignation to the variable .

```
Int v_d =20;
```

J . declaration values----> defines variable data type and name .

```
class Demo{
    int x;
    public Demo(){
        x = 10;
    }
    public static void main ( String args[] ){
```

```

        Demo myObj = new Demo();
        System.out.println(" x = "+myObj.x);
    }
}

```

K. constructor ----->A block of code which is automatically called when a new object of a class is created .

```

{
    System.out.println("V7 Software Industry");
}

```

L. Instance Blocks-----> Unnamed block of code defied inside a class.

```

class Demo{
    static int x = 10;////---static variable---
    public static void main (String args[]){
        int y = x + 10;
        System.out.println(y);
    }
}

```

M.Static variables-----> variables that can access in whole program but inside the class.

```

class Demo{
    int x = 10;////---instance variable---
    public static void main (String args[]){

    }
}

```

N Instance variables ---->Variables that created inside the class but outside the methods .

```

////////////////////////////////////

```

2

D

```

////////////////////////////////////

```

3

Output ---> 89;

```

////////////////////////////////////

```

4

In oop, attributes represent the state or data of an object,while method represent the behavior or action that the object can perform.

```

////////////////////////////////////

```

5

Illllegal lines -----> 2,3,4

reason -----> instance variables are not called by the object that has created.

```

////////////////////////////////////

```

6

Output

Volume : 0

length of box : 0

width of box : 0

height of box : 0

Reason ---> values are not passed to the method.

////////////////////////////////////

7

Output

Volume : 180

length of box : 12

width of box : 5

height of box : 3

////////////////////////////////////

8

Output

default constructor

length of box : 2

width of box : 2

height of box : 2

Reason ---> values are assigned to the instance variables through the constructor

////////////////////////////////////

9

Output

Parameterized constructor

Volume : 60

Parameterized constructor

Volume : 180

////////////////////////////////////

10

Compile error when print width of the box because of the encapsulation .

////////////////////////////////////

11

Illegal lines ---> 4,5,6

Reason because the haven't identifiers.

////////////////////////////////////

12

To create objects we need a class . without a class we can not create objects.

```
class Demo{
    int x = 10;
    public static void main (String args[]){
```

```
        Demo b1 = new Demo(); //this is the time we need a class to create objects
```

```
        System.out.println(b1.x);
```

```
    }
```

```
}
```

////////////////////////////////////

13

#Constructor

```
#method
class Demo{
    static void printx(){
        int x = 12;
        System.out.println("x:"+x);
    }
    public static void main(String args[]){
        printx();//should call manually
    }
}
```

////////////////////////////////////

```
16
C
////////////////////////////////////
17
```

```
A,B,D
////////////////////////////////////
```

```
18
Out put
100 101
0 0
////////////////////////////////////
19
```

Output ----> 3001

```
////////////////////////////////////
20
Output --->2001
```

' this ' keyword give the current object instance on which the method is being called.
Therefore the output is 2001
////////////////////////////////////

```
21
D
////////////////////////////////////
22
A and D
```

Reason for A
From the the created object 100 pass to the constructor ' l ' and l is assigned to the 'a'.

```
////////////////////////////////////
23
Encapsulation is processes of hiding the data from outsiders. We can create a encapsulated  
Variable from putting ' private ' in-front of the attribute .
```

```
class Box{
    private int length;//Encapsulation
    private int width;//Encapsulation
    private int height;//Encapsulation

    Box(int a , int b, int c){
        length = a;
        width = b;
        height = c;
    }
    void printVolume(){
        int volume = length*width*height;
        System.out.println("Volume of the box : "+volume);
    }
}
class Demo{
    public static void main (String args[]){
```

```

    }
}
////////////////////////////////////
24

```

In Week Encapsulation , variables are exposed using public , protected or package-private.

```
class Test{
    private int x;

    public void setX(int x){
        this.x = x;
    }
    public int getX(){
        return x;
    }
}

class Demo{
    public static void main(String args[]){
        Test ob1 = new Test();
        ob1.setX(10);

        System.out.println(ob1.getX());
    }
}
```

```
class Test{
    public int x;

    Test (int i){
        x =i;
    }
}

class Demo{
    public static void main(String args[]){
        Test ob1 = new Test(12);

        System.out.println(ob1.x);
    }
}
```

////////////////////////////////////

25
D
26

////////////////////////////////////

```
class Date{
    private int year = 1970;
    private int month = 1;
    private int day = 1;

    Date(int year, int month, int day){
        this.year=year;
        this.month=month;
        this.day=day;
    }

    void printDate(){
        System.out.println("Date is : "+year+" : "+month+" : "+day);
    }

    public void setYear(int year){
        this.year=year;
    }
    public void setMonth(int month){
        this.month=month;
    }
    public void setDay(int day){
        this.day=day;
    }
    public int getYear(){
        return year;
    }
    public int getMonth(){
        return month;
    }
    public int getDay(){
        return day;
    }
}

class Demo{
    public static void main (String args[]){
        Date d1 = new Date(2016,5,30);
        d1.printDate();

        d1.setYear(2016);
        d1.setMonth(5);
        d1.setDay(31);

        System.out.println("Year :"+d1.getYear());
        System.out.println("Month :"+d1.getMonth());
    }
}
```

```
System.out.println("Day :"+d1.getDay());
```

```
    }  
}  
/////////////////////////////////////  
27  
Line 1-->Declares a reference variable named c1 of thpe customer.  
Line 2-->Creates a new Customer  object  
Line 3-->Calls the printCustomer() method of the object referenced by c1.  
Line 4--->Declare another Customer reference variable named c2.  
Line 5--->Creates a new Customer object using the default constructor.  
Line 6--->Calls the method  
Line 7--->Pints the details stored in c2.  
Line 8--->Declares a third Customer reference variable named c3.  
Line 9--->Creates a new Customer object using the default constructor.  
Line 10--->Calls the method setId.  
Line 11---->Calls the setName() method  
Line 12---->prints c3.  
/////////////////////////////////////  
28  
All lin1 ,2,3 and 4 are illegal  
/////////////////////////////////////  
29  
F  
/////////////////////////////////////  
30  
class Rectangle{  
    private double length;  
    private double width;  
  
    Rectangle(){  
        length = 1.0;  
        width=1.0;  
    }  
  
    public void setLength(double length){  
        if(length > 0.0 && length < 20.0){  
            this.length=length;  
        }else{  
            System.out.println("Invalid length");  
        }  
    }  
    public void setWidth(double width){  
        if (width > 0.0 && width < 20.0){  
            this.width=width;  
        }else{  
            System.out.println("Invalid width");  
        }  
    }  
  
    public double getLength(){  
        return length;  
    }  
    public double getWidth(){  
        return width;  
    }  
}
```



```

        public double area(){
            return length*width;
        }
        public double perimeter(){
            return 2*(length+width);
        }
    }
}
class Demo{
    public static void main (String args[]){

        Rectangle r1 = new Rectangle();

        System.out.println("Default Length: " + r1.getLength());
        System.out.println("Default Width: " + r1.getWidth());

        r1.setLength(10.5);
        r1.setWidth(5.5);

        System.out.println("\nUpdated Length: " + r1.getLength());
        System.out.println("Updated Width: " + r1.getWidth());

        System.out.println("Area: " + r1.area());
        System.out.println("Perimeter: " + r1.perimeter());

    }
}

```

//

31

Compile error at line 1 and 3

//

32

F

//

33

Line 5 and line 8

//

34

Static variables can be access through out the program but instance variables access
Only in the class.

//

35

Line 1,4,6 and 9 are cause compile errors

//

36

//

37

Static variables can be access through out the program but instance variables access
Only in the class.

//

38

Output---->Box is loaded into memory

Reason --->Because of the static block java JVM automatically execute the the block

//

39

A box object is created..
A box object is created..

Reason ---> Because two objects are created

-

////////////////////////////////////

40

```
class Cylinder{
    private int length;
    private int radius;
    Cylinder(){
        length = 1;
        radius = 1;
    }
    Cylinder(int l, int r){
        length = l;
        radius = r;
    }
    public void setLength(int length){
        this.length=length;
    }
    public void setRadius(int radius){
        this.radius=radius;
    }
    public int getLength(){
        return length;
    }
    public int getRadius(){
        return radius;
    }
    public int area(){
        int pi = 22/7;
        return 2*pi*radius*(radius+length);
    }
    public int volume(){
        int pi = 22/7;
        return pi*radius*radius*length;
    }
}

class Demo{
    public static void main (String args[]){
        Cylinder c1 = new Cylinder(1,1);

        c1.setLength(1);
        c1.setRadius(1);

        System.out.println("Area: " + c1.area());
        System.out.println("Volume: " + c1.volume());

    }
}
```

```
}
```

```
////////////////////////////////////  
41
```

Output

Box is loaded into memory
A box object is created..
A box object is created..
A box object is created..

'A box object is created..' is printed because it's a instance block so automatically
Printing

```
////////////////////////////////////  
42  
Line 3 and 4 are illegal
```

```
////////////////////////////////////  
43  
A ---> 1 2 3  
B ---->1 2 3 2 3  
C ---> 1 4  
D- -->compile error  
E ----> 1 4  
F print nothing
```

```
////////////////////////////////////  
44
```

```
////////////////////////////////////  
45
```

#Methods call by values
a copy of the actual value is passed to the method.
#Methods call by reference
the reference of the original object is passed to the method.

```
////////////////////////////////////  
46
```

Line 1--->legal
Line 2--->legal
Line 3--->illegal
Line 4--->legal
Line 5-->illlegal
Line 6--->legal

```
////////////////////////////////////  
47
```

```
////////////////////////////////////  
48
```

```
class Employee {  
    private String firstName;  
    private String lastName;  
    private double monthlySalary;
```

```
    public Employee(String firstName, String lastName, double monthlySalary) {  
        this.firstName = firstName;  
        this.lastName = lastName;
```

```

        if (monthlySalary > 0) {
            this.monthlySalary = monthlySalary;
        }
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public void setMonthlySalary(double monthlySalary) {
        if (monthlySalary > 0) {
            this.monthlySalary = monthlySalary;
        }
    }

    public String getFirstName() {
        return firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public double getMonthlySalary() {
        return monthlySalary;
    }

    public double getYearlySalary() {
        return monthlySalary * 12;
    }
}

class Demo {
    public static void main(String[] args) {

        Employee emp1 = new Employee("John", "Smith", 5000);
        Employee emp2 = new Employee("Mary", "Jones", 6000);

        System.out.println("Employee 1: " + emp1.getFirstName() + " " + emp1.getLastName());
        System.out.println("Yearly Salary = " + emp1.getYearlySalary());

        System.out.println();

        System.out.println("Employee 2: " + emp2.getFirstName() + " " + emp2.getLastName());
        System.out.println("Yearly Salary = " + emp2.getYearlySalary());

        emp1.setMonthlySalary(emp1.getMonthlySalary() * 1.10);
    }
}

```

```

emp2.setMonthlySalary(emp2.getMonthlySalary() * 1.10);

System.out.println("\nAfter 10% Raise:");

System.out.println("Employee 1: " + emp1.getFirstName() + " " + emp1.getLastName());
System.out.println("Yearly Salary = " + emp1.getYearlySalary());

System.out.println();

System.out.println("Employee 2: " + emp2.getFirstName() + " " + emp2.getLastName());
System.out.println("Yearly Salary = " + emp2.getYearlySalary());
}
}

```

```

////////////////////////////////////

```

49

code : 1001

code : 1001

```

////////////////////////////////////

```

50

```

////////////////////////////////////

```

51

D

```

////////////////////////////////////

```

52

Output

MyClass()

MyClass(int,int)

MyClass(int)

```

////////////////////////////////////

```

53

class Box{

private int length;

private int width;

private int height;

public Box(){

length = 0;

width = 0;

height = 0;

}

public Box(int length, int width, int height){

this.length = length;

this.width = width;

this.height = height;

}

public Box(int side){

this.length = side;

this.width = side;

this.height = side;

}

```

public Box(Box b){
    this.length = b.length;
    this.width = b.width;
    this.height = b.height;
}

public void setLength(int length){
    this.length = length;
}

public void setWidth(int width){
    this.width = width;
}

public void setHeight(int height){
    this.height = height;
}

public void setDimension(int length, int width, int height){
    this.length = length;
    this.width = width;
    this.height = height;
}

public void setDimension(int side){
    this.length = side;
    this.width = side;
    this.height = side;
}

public void setDimension(Box b){
    this.length = b.length;
    this.width = b.width;
    this.height = b.height;
}

public int getVolume(){
    return length * width * height;
}

public void printVolume(){
    System.out.println("Volume = " + getVolume());
}

public Box getCopy(){
    return new Box(this);
}
}

```

```
class Demo{
    public static void main(String args[]){
        Box b1 = new Box();
        b1.setLength(12);
        b1.setWidth(5);
        b1.setHeight(3);
        b1.printVolume();

        b1.setDimension(120,50,30);
        System.out.println("Volume " + b1.getVolume());

        Box b2 = new Box(4,2,3);
        b2.printVolume();

        Box b3 = new Box(b2);
        b3.printVolume();

        Box b4 = new Box(10);
        b4.printVolume();

        Box b5 = new Box();
        b5.setDimension(12);
        b5.printVolume();

        Box b6 = new Box();
        b6.printVolume();

        b6.setDimension(b1);
        b6.printVolume();

        Box b7 = b3.getCopy();
        b7.printVolume();
    }
}
```